	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Aldo Hernandez Fuentes

Asignatura: Modelos De Programacion Orientada a Objetos

Grupo: 7

No de Práctica(s): Practica NO. 8

Integrante(s): TORRES SANCHEZ EFREN

GONZAGA VARGAS DANIEL WILFRIDO

No. de lista o brigada: 13

Semestre: 2027/01

Fecha de entrega: 25/10/2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:
Implementar programas que permitan crear objetos con diferentes valores iniciales a lo largo de una jerarquía de clases.

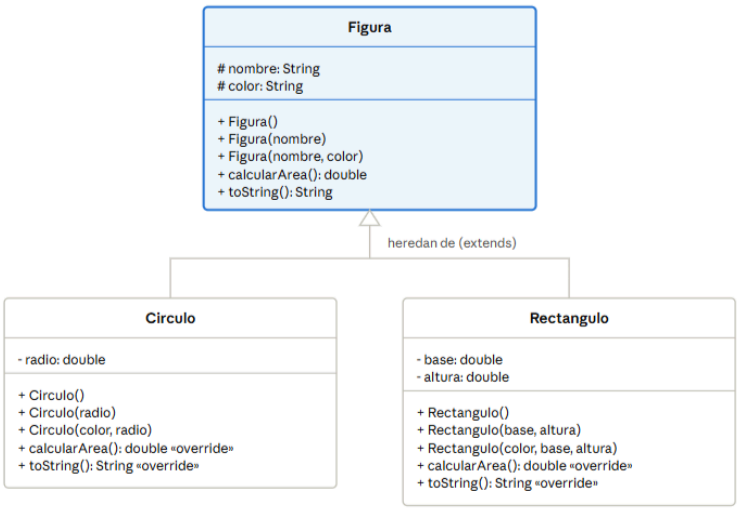
Ejercicio 1:

Descripcion del problema: El problema consiste en crear en Java una clase Figura y derivar de ella las clases Circulo y Rectangulo, cada una con sus propios constructores. Para lograrlo, los constructores deben reutilizarse entre sí con this, y cada subclase debe aprovechar el constructor de Figura mediante super.

Diseño de la solucion:
Para este ejercicio se crearon tres clases. La clase Figura es la superclase y contiene los atributos nombre y color, declarados como protected para que las subclases puedan usarlos. Tiene tres constructores sobrecargados que se encadenan con this: el constructor vacío llama al que recibe el nombre, enviando "sin-nombre"; ese, a su vez, llama al constructor completo, enviando "blanco" como color. Solo el constructor completo asigna los valores e imprime un mensaje. Además, define el método calcularArea(), que en una figura genérica devuelve 0, y sobrescribe toString() para mostrar el nombre y el color.

La clase Circulo hereda de Figura y agrega el atributo propio radio. También cuenta con tres constructores: el vacío crea un círculo de radio 1, el que recibe el radio usa el color blanco por defecto, y el completo recibe el color y el radio. Este último llama a super("circulo", color) para inicializar la parte heredada y después asigna el radio. Sobrescribe calcularArea() con la fórmula $\pi \cdot r^2$ y toString() para agregar el radio y el área.

La clase Rectangulo sigue la misma estructura con los atributos propios base y altura. Su constructor vacío crea un rectángulo de 1 × 1, el segundo recibe solo las medidas y el completo recibe además el color. Sobrescribe calcularArea() con base·altura y toString() para agregar sus medidas y su área.



Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio1, donde cada clase se encuentra en su propio archivo: Figura.java, Circulo.java, Rectangulo.java y Main.java. Al compilar con javac se genera un archivo .class por cada clase. La clase Main contiene el método main y funciona como punto de entrada del programa; ahí se crean figuras usando distintos constructores para comprobar que todos funcionan.

Fragmentos relevantes del codigo:

- **Constructores encadenados con this:** En Figura, cada constructor con menos parámetros llama a otro de la misma clase mediante this(...), completando los datos que faltan con valores por defecto. Así, toda la inicialización se escribe una sola vez, en el constructor completo.

```
public Figura() {
    this("sin-nombre");
}

public Figura(String nombre) {
    this(nombre, "blanco");
}

public Figura(String nombre, String color) {
    this.nombre = nombre;
    this.color = color;
    System.out.println("[Figura] creando " + nombre);
}
```

- **Constructores de la subclase con this y super:** Circulo encadena sus propios constructores con this hasta llegar al completo, el cual llama a super() para que Figura inicialice el nombre y el color. La llamada a super() debe ser la primera instrucción del constructor.

```
public Circulo() {
   this(1.0);
}

public Circulo(double radio) {
   this("blanco", radio);
}

public Circulo(String color, double radio) {
   super("circulo", color);
   this.radio = radio;
   System.out.println("[Circulo] listo con radio " + radio);
}
```

- **Sobrescritura de métodos**: Cada subclase redefine calcularArea() con su propia fórmula. En toString() se reutiliza el texto que genera Figura con super.toString() y solo se agregan los datos propios.

```
@Override
public double calcularArea() {
   return Math.PI * radio * radio;
}

@Override
public String toString() {
   return super.toString() + " | radio " + radio
      + " | area " + String.format("%.2f", calcularArea());
}
```

Funcionamiento general del programa:

Al ejecutar el programa, el método main crea cinco objetos: una figura genérica con el constructor vacío, un círculo con valores por defecto, un círculo rojo de radio 2.5, un rectángulo de 4 × 3 y un rectángulo azul de 6 × 2. Cada vez que se crea un círculo o un

rectángulo, primero aparece el mensaje de Figura y después el de la subclase, porque el constructor del hijo llama a super() antes de ejecutar su propio código. Una vez creados, se imprime cada objeto, lo que llama automáticamente a su toString(): la figura genérica solo muestra su nombre y color, mientras que el círculo y el rectángulo muestran además sus medidas y su área calculada con su propia fórmula.

Pruebas y Resultados:

```
> javac *.java
> java Main
[Figura] creando sin-nombre
[Figura] creando circulo
[Circulo] listo con radio 1.0
[Figura] creando circulo
[Circulo] listo con radio 2.5
[Figura] creando rectangulo
[Rectangulo] listo de 4.0 x 3.0
[Figura] creando rectangulo
[Rectangulo] listo de 6.0 x 2.0
sin-nombre | color blanco
circulo | color blanco | radio 1.0 | area 3.14
circulo | color rojo | radio 2.5 | area 19.63
rectangulo | color blanco | 4.0 x 3.0 | area 12.00
rectangulo | color azul | 6.0 x 2.0 | area 12.00
```

Ejercicio 2:

Descripcion del problema: El problema consiste en implementar en Java una jerarquía de figuras geométricas formada por Triángulo, Cuadrado, Rectángulo, Círculo y Pentágono, en la que cada subclase calcule su propia área. Todas las figuras comparten un nombre y un color, pero cada una necesita medidas distintas y una fórmula diferente para obtener su área. Además, se debe aprovechar la herencia para no repetir código entre figuras que están relacionadas, como el cuadrado y el rectángulo.

Diseño de la solucion

La clase Figura es la superclase de toda la jerarquía. Contiene los atributos protected nombre y color, tres constructores encadenados con this y el método calcularArea(), que devuelve 0 porque una figura genérica no tiene medidas. Su toString() muestra el nombre, el color y el área; como llama a calcularArea(), cada figura muestra automáticamente el área calculada con su propia fórmula.

De Figura heredan directamente cuatro clases, cada una con sus atributos propios y su fórmula de área:

Clase	Atributos Propios	Area
Triangulo	Base, Altura	b*h/2
Rectangulo	Base, Altura	b*h
Circulo	Radio	Pi(r2)
Pentagono	Lado, Apotema	(5*lado)apotema/2

La clase Cuadrado no hereda directamente de Figura, sino de Rectangulo, porque un cuadrado es un rectángulo cuya base y altura son iguales. De esta forma, Cuadrado no necesita atributos propios ni volver a escribir calcularArea(): le basta con enviar el mismo lado como base y como altura al constructor de Rectangulo. Esto forma una herencia de varios niveles: Figura → Rectangulo → Cuadrado.

Cada subclase tiene dos constructores: uno que recibe solo las medidas y usa el color blanco por defecto, y otro completo que recibe también el color. El primero llama al segundo con this, y el segundo llama a la superclase con super.

Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio2, con un archivo por clase: Figura.java, Triangulo.java, Rectangulo.java, Cuadrado.java, Circulo.java, Pentagono.java y Main.java. Al compilar se genera un archivo .class por cada una. La clase Main contiene el método main, donde se crean las cinco figuras y se calcula el área total.

Fragmentos relevantes del codigo:

- **Constructor protegido en Rectangulo:** Además de sus dos constructores públicos, Rectangulo tiene un constructor protected que también recibe el nombre. Las clases externas no pueden usarlo, pero Cuadrado sí, y así puede llamarse "cuadrado" en lugar de "rectangulo".

```
public Rectangulo(String color, double base, double altura) {
   this("rectangulo", color, base, altura);
}
```

```
protected Rectangulo(String nombre, String color,
        double base, double altura) {
    super(nombre, color);
    this.base = base;
    this.altura = altura;
    System.out.println("[Rectangulo] " + base + " x " + altura);
}
```

- **Herencia de varios niveles:** Cuadrado hereda de Rectangulo y en su constructor envía el mismo lado como base y como altura. No sobrescribe calcularArea(), porque la fórmula base · altura que hereda ya da lado · lado.

```
public class Cuadrado extends Rectangulo {

    public Cuadrado(String color, double lado) {
        super("cuadrado", color, lado, lado);
        System.out.println("[Cuadrado] lado " + lado);
    }
}
```

- **Cálculo del área en cada subclase:** Cada figura sobrescribe calcularArea() con su fórmula. En el pentágono, primero se obtiene el perímetro y después se multiplica por la apotema y se divide entre 2.

```
@Override
public double calcularArea() {
    double perimetro = 5 * lado;
    return perimetro * apotema / 2;
}
```

- **Recorrido de la jerarquía:** En Main, un arreglo de tipo Figura guarda objetos de las cinco subclases. Al llamar a calcularArea() sobre cada elemento, Java ejecuta la fórmula de la figura real, y las áreas se acumulan en un total.

```
Figura[] figuras = new Figura[5];
figuras[0] = new Triangulo(6, 4);
figuras[1] = new Cuadrado("verde", 5);
figuras[2] = new Rectangulo("azul", 8, 3);
figuras[3] = new Circulo("rojo", 2);
```

```
figuras[4] = new Pentagono(4, 2.75);

double areaTotal = 0;
for (Figura f : figuras) {
    System.out.println(f);
    areaTotal += f.calcularArea();
}
```

Funcionamiento general del programa:

Al ejecutar el programa, el método main crea un arreglo de tipo Figura con cinco elementos: un triángulo de base 6 y altura 4, un cuadrado verde de lado 5, un rectángulo azul de 8 × 3, un círculo rojo de radio 2 y un pentágono de lado 4 y apotema 2.75. Durante la creación, cada constructor imprime un mensaje; en el caso del cuadrado aparecen tres mensajes seguidos, de Figura, Rectangulo y Cuadrado, lo que muestra cómo se ejecutan los constructores de toda la cadena de herencia. Después, un ciclo for recorre el arreglo, imprime cada figura con su área y la suma al total. Al final se muestra el área total de las cinco figuras, 101.07.

Pruebas y Resultados:

```
> javac *.java
> java Main
[Figura] creando triangulo
[Triangulo] base 6.0, altura 4.0
[Figura] creando cuadrado
[Rectangulo] 5.0 x 5.0
[Cuadrado] lado 5.0
[Figura] creando rectangulo
[Rectangulo] 8.0 x 3.0
[Figura] creando circulo
[Circulo] radio 2.0
[Figura] creando pentagono
[Pentagono] lado 4.0, apotema 2.75
----- Figuras -----
triangulo | color blanco | area 12.00 | b = 6.0, h = 4.0
cuadrado | color verde | area 25.00 | 5.0 x 5.0 (lado 5.0)
rectangulo | color azul | area 24.00 | 8.0 x 3.0
circulo | color rojo | area 12.57 | r = 2.0
pentagono | color blanco | area 27.50 | lado 4.0, apotema 2.75
Area total: 101.07
```

Ejercicio 3:

Descripcion del problema: El problema consiste en diseñar en Java las clases de un videojuego: Personaje, Mago, Guerrero, Villano y Aldeano, reutilizando código mediante herencia y constructores. Todos los personajes del juego tienen un nombre, un nivel y una cantidad de vida, y pueden atacar y recibir daño. Sin embargo, cada tipo de personaje tiene características propias y ataca de una forma distinta, por lo que las subclases deben agregar sus atributos y redefinir el comportamiento del ataque.

Diseño de la solucion:

La clase Personaje es la superclase y reúne lo que tienen en común todos los personajes. Sus atributos son nombre, nivel y vida, declarados como protected. Tiene tres constructores encadenados con this: el vacío usa el nombre "sin-nombre", el que recibe el nombre asigna nivel 1 y 100 de vida, y el completo recibe los tres valores e imprime un mensaje. Sus métodos son atacar(), que devuelve un daño base igual al doble del nivel; recibirDanio(), que resta vida sin permitir que baje de 0; estaVivo(), getNombre() y toString().

Clase	Atributos Propios	Como ataca
Mago	Mana, Hechizo	Daño base + 25, gastando 20 de maná
Guerrero	Fuerza, Arma	Daño base + fuerza
Villano	Maldad, Guarida	Daño base + maldad / 10
Aldeano	Oficio	No pelea, su daño es 0

Mago, Guerrero y Villano tienen dos constructores: uno que recibe solo el nombre y completa el resto con valores por defecto mediante this, y uno completo que llama a super(nombre, nivel, vida). Aldeano, en cambio, aprovecha el constructor de Personaje que recibe solo el nombre, con super(nombre), para heredar el nivel 1 y los 100 puntos de vida por defecto. Además, tiene el método propio ofrecerAyuda().

Todas las subclases sobrescriben atacar() a partir de super.atacar(), es decir, parten del daño base del personaje y le aplican su propia regla. También sobrescriben toString() para anteponer su tipo y agregar sus atributos propios.

Implementacion del proyecto:

El proyecto está organizado en la carpeta Ejercicio3, con un archivo por clase: Personaje.java, Mago.java, Guerrero.java, Villano.java, Aldeano.java y Main.java. Al compilar se genera un archivo .class por cada una. La clase Main contiene el método main, donde se crean los personajes, se muestran sus datos y se simula una pequeña batalla contra el villano.

Fragmentos relevantes del código:

- **Reutilización de constructores en las subclases:** El constructor de Guerrero que recibe solo el nombre llama con this al constructor completo, enviándole los valores de un guerrero novato. El constructor completo, a su vez, usa super() para que Personaje inicialice el nombre, el nivel y la vida.

```
public Guerrero(String nombre) {
   this(nombre, 1, 150, 10, "Espada");
}

public Guerrero(String nombre, int nivel, int vida,
```

```
        int fuerza, String arma) {
    super(nombre, nivel, vida);
    this.fuerza = fuerza;
    this.arma = arma;
    System.out.println("[Guerrero] empuna " + arma);
}
```

- **Sobrescritura del ataque:** Cada personaje redefine atacar() partiendo del daño base que calcula Personaje con super.atacar(). El Mago, por ejemplo, solo hace daño extra si tiene suficiente maná, y cada vez que lo usa se le descuenta.

```
@Override
public int atacar() {
    if (mana >= 20) {
        mana = mana - 20;
        return super.atacar() + 25;
    }
    return super.atacar();
}
```

Uso de los personajes en Main: Se declara un arreglo de tipo Personaje que guarda objetos de las cuatro subclases. Al recorrerlo, cada objeto se imprime con su propio toString(). Como ofrecerAyuda() solo existe en Aldeano, se verifica el tipo con instanceof y se hace un cast antes de llamarlo.

```
for (Personaje p : personajes) {
    System.out.println(p);
    if (p instanceof Aldeano) {
        Aldeano a = (Aldeano) p;
        System.out.println("  -> " + a.ofrecerAyuda());
    }
}
```

Funcionamiento general del problema: Al ejecutar el programa, el método main crea un arreglo de tipo Personaje con cinco elementos: el mago Merlin, el guerrero Arturo, la villana Morgana y dos aldeanos, uno llamado Tomás que es herrero y otro creado con el constructor vacío. Al crear cada uno se imprimen primero el mensaje de Personaje y después el de su subclase. A continuación se muestran los datos de todos los personajes y los aldeanos ofrecen su ayuda. Después inicia la batalla: Merlin, Arturo y

Tomás atacan a Morgana, y cada uno le hace un daño diferente según su forma de atacar: 31, 12 y 0, respectivamente. Luego Morgana contraataca a Arturo y le quita 15 puntos de vida. Al final se muestra el estado de ambos: Morgana queda con 157 de vida y Arturo con 135.

Conclusiones

La práctica permitió comprender la diferencia entre this y super dentro de un constructor. Con this(...) un constructor llama a otro de la misma clase, lo que permite crear objetos con valores por defecto sin repetir el código de inicialización. También se reforzó la idea de sobrescribir métodos a partir de la versión del padre con super.metodo(), como en atacar() o toString(), de modo que cada subclase solo agrega lo que la hace diferente.

Pruebas y Resultados:

```
> javac *.java
> java Main
[Personaje] creando Merlin
[Mago] domina el hechizo Bola de fuego
[Personaje] creando Arturo
[Guerrero] empuna Espada
[Personaje] creando Morgana
[Villano] se esconde en Cueva oscura
[Personaje] creando Tomas
[Aldeano] trabaja de herrero
[Personaje] creando Aldeano
[Aldeano] trabaja de sin oficio
----- Personajes -----
Mago: Merlin | nivel 3 | vida 90 | mana 100 | hechizo Bola de fuego
Guerrero: Arturo | nivel 1 | vida 150 | fuerza 10 | arma Espada
Villano: Morgana | nivel 5 | vida 200 | maldad 50 | guarida Cueva oscura
Aldeano: Tomas | nivel 1 | vida 100 | oficio herrero
  -> Tomas (herrero) ofrece ayuda a los heroes.
Aldeano: Aldeano | nivel 1 | vida 100 | oficio sin oficio
  -> Aldeano (sin oficio) ofrece ayuda a los heroes.
----- Batalla -----
Merlin ataca a Morgana y le hace 31 de danio
Arturo ataca a Morgana y le hace 12 de danio
Tomas ataca a Morgana y le hace 0 de danio
Morgana contraataca a Arturo y le hace 15 de danio
----- Estado final -----
Villano: Morgana | nivel 5 | vida 157 | maldad 50 | guarida Cueva oscura
Guerrero: Arturo | nivel 1 | vida 135 | fuerza 10 | arma Espada
```

Referencias:

- Oracle. (s. f.). *Inheritance (The Java™ Tutorials)*. Oracle. https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html
- Oracle. (s. f.). *Java documentation*. Oracle. https://docs.oracle.com/en/java/
- Deitel, P., & Deitel, H. (2017). *Java: How to program* (11th ed.). Pearson.